```
print("Hello! I am your smart-search helper. 🤖")
print("Give me a puzzle and I will find the shortest answer with A*! ⭐")
def astar(start, goal, get_neighbours, guess):
    # The to-do list. Each note is [f, g, place, path_so_far].
    frontier = [[guess(start), 0, start, [start]]]
    visited = []            # places we already explored
    explored_count = 0      # how many places we explored (for comparing)

    while len(frontier) > 0:
        # 1) Find the note with the SMALLEST f by looking through the list.
        best = 0
        for i in range(len(frontier)):
            if frontier[i][0] < frontier[best][0]:
                best = i
        note = frontier.pop(best)     # take that note out of the list

        f, g, place, path = note      # unpack the note

        # 2) If we already explored this place, skip it.
        if place in visited:
            continue
        visited.append(place)
        explored_count = explored_count + 1

        # 3) Did we reach the goal? Then we are done.
        if place == goal:
            return path, explored_count

        # 4) Add each neighbour to the to-do list as a new note.
        for nxt in get_neighbours(place):
            if nxt not in visited:
                new_g = g + 1                       # one more step
                new_f = new_g + guess(nxt)          # f = g + h
                frontier.append([new_f, new_g, nxt, path + [nxt]])

    return None, explored_count     # no path found

print("A* is ready. ✅")
grid = [
    "..........",
    "....###....",
    "S........G",
    "....###....",
    "..........",
]

def find(letter):
    for r in range(len(grid)):
        for c in range(len(grid[r])):
```

```
                if grid[r][c] == letter:
                    return (r, c)

    start = find("S")
    goal  = find("G")
    rows, cols = len(grid), len(grid[0])

    print("Start S is at", start)
    print("Goal  G is at", goal)
    print()
    for line in grid:
        print("  " + "  ".join(line))
    def grid_neighbours(box):
        r, c = box
        out = []
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:   # up, down, left, right
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:           # stay on the grid
                if grid[nr][nc] != "#":                     # not a wall
                    out.append((nr, nc))
        return out

    print("From the start", start, "the robot can step to:", grid_neighbours(start))
    # Guess 1: no hint at all
    def grid_zero(box):
        return 0

    # Guess 2: the Manhattan distance (rows apart + columns apart)
    def grid_manhattan(box):
        r, c = box
        return abs(r - goal[0]) + abs(c - goal[1])

    print("Manhattan guess from the start:", grid_manhattan(start))
    path_zero, explored_zero = astar(start, goal, grid_neighbours, grid_zero)
    path_manh, explored_manh = astar(start, goal, grid_neighbours, grid_manhattan)

    print("Guess          Path length    Boxes explored")
    print("-" * 46)
    print("Zero (no hint) ", len(path_zero) - 1, "steps      ", explored_zero)
    print("Manhattan      ", len(path_manh) - 1, "steps      ", explored_manh)
    print()
    print("Same shortest path. But Manhattan explores FEWER boxes. ⭐")
    def show_grid_path(path):
        path_set = set(path)
        for r in range(rows):
            line = ""
            for c in range(cols):
                if (r, c) == start:
                    line += " S "
                elif (r, c) == goal:
                    line += " G "
```

```
            elif (r, c) in path_set:
                line += " * "
            else:
                line += " " + grid[r][c] + " "
        print(line)

print("The robot's path:")
print()
show_grid_path(path_manh)
goal_state  = "123456780"     # the tidy goal (0 is the empty space)
start_state = "123450678"     # a scrambled start

def show_puzzle(state):
    for r in range(3):
        row = state[r*3 : r*3 + 3]
        row = row.replace("0", "_")          # show the empty space as _
        print("   " + "  ".join(row))

print("Start:")
show_puzzle(start_state)
print()
print("Goal:")
show_puzzle(goal_state)
def puzzle_neighbours(state):
    out = []
    zero = state.index("0")          # where the empty space is
    r, c = zero // 3, zero % 3
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:          # stay inside the box
            new_zero = nr * 3 + nc
            tiles = list(state)
            # swap the empty space with the tile next to it
            tiles[zero], tiles[new_zero] = tiles[new_zero], tiles[zero]
            out.append("".join(tiles))
    return out

print("From the start, we can reach these arrangements:")
for s in puzzle_neighbours(start_state):
    print("  ", s)
# Guess 1: no hint
def puzzle_zero(state):
    return 0

# Guess 2: count tiles that are NOT in the right place (ignore the empty space)
def puzzle_wrong_tiles(state):
    count = 0
    for i in range(9):
        if state[i] != "0" and state[i] != goal_state[i]:
            count = count + 1
    return count
```

```
print("Wrong tiles in the start:", puzzle_wrong_tiles(start_state))
ans_zero,  count_zero  = astar(start_state, goal_state, puzzle_neighbours, puzz
ans_smart, count_smart = astar(start_state, goal_state, puzzle_neighbours, puzz

print("Guess            Moves to solve    Arrangements explored")
print("-" * 56)
print("Zero (no hint)  ", len(ans_zero) - 1,  "moves          ", count_zero)
print("Wrong-tiles     ", len(ans_smart) - 1, "moves          ", count_smart)
print()
print("Same number of moves. But the smart guess explores FAR fewer arrangement
print("Solving the puzzle, one slide at a time:")
print()
for step, state in enumerate(ans_smart):
    print("Move", step, ":")
    show_puzzle(state)
    print()
```

```
   5  _  6
   4  7  8

Move 10 :
   1  2  3
   _  5  6
   4  7  8

Move 11 :
   1  2  3
   4  5  6
   _  7  8

Move 12 :
   1  2  3
   4  5  6
   7  _  8

Move 13 :
   1  2  3
   4  5  6
   7  8  _
```

Start coding or generate with AI.